SIMATS SCHOOL OF ENGINEERING

Approved By AICTE | IET-UK Accreditation

Name : V.V.D.Nagasai Ratan Reddy	**Reg. no. 192324172**
Course Code: DSA03	**Course Name: Natural Language Processing**
Branch: AI&DS	**Year : 2023**

Assignment 3

1. **Draw tree structures for the following ATIS sentences:** (10M) (CO3) (BLT4)
   a. **Does American airlines have a flight between five a.m. and six a.m.**
   b. **I would like to fly on American airlines.**
   c. **Please repeat that.**
   d. **Does American 487 have a first class section?**
   e. **I need to fly between Philadelphia and Atlanta.**
   f. **What is the fare from Atlanta to Denver?**
   g. **Is there an American airlines flight from Philadelphia to Dallas?**

2. **Consider the below grammar to construct the Early parser for the input string "Christ Opened the eyes".** (10M) (CO3) (BLT4)

   S -> NP VP VP -> V NP PP V -> opened
   NP -> N PP -> Prep NP V -> ate
   NP -> Det NP N -> Christ Det -> the
   NP -> NP NP N -> blind Det -> a
   VP -> V N -> eyes Prep -> of VP -> V NP N -> bread Prep -> in

Introduction to ATIS Tree Structures

ATIS stands for Air Travel Information System. ATIS-style sentences are short spoken or typed requests about flights, airlines, fares, routes, times, and travel preferences. In Natural Language Processing, a syntactic parser analyzes the grammatical structure of such a sentence and represents relationships among words and phrases.

A tree structure, commonly called a parse tree, shows how a sentence is built from grammatical constituents. The root is usually S (Sentence). Internal nodes represent phrases such as NP (Noun Phrase), VP (Verb Phrase), PP (Prepositional Phrase), and other categories. Leaf nodes are the actual words in the sentence.

The seven sentences in this question are parsed using a conventional phrase-structure grammar. The trees are presented in bracketed form and ASCII form so that they can be copied into a handwritten or drawn diagram. The exact labels may vary slightly across textbooks, but the constituent relationships are made explicit.

Tree / structural representation

> Parsing convention: S = sentence; NP = noun phrase; VP = verb phrase; PP = prepositional phrase; Det = determiner; N = noun; V = verb; Aux = auxiliary; Pron = pronoun; Adv = adverb; Adj = adjective; P = preposition.

To verify a tree, read its leaf nodes from left to right. The resulting sequence must reproduce the original sentence exactly. Every internal node should dominate the words that form its constituent, and phrase boundaries should reflect the intended grammatical relationships.

When drawing the tree by hand, keep category labels aligned and avoid crossing branches. Group multiword names under a single NP where appropriate, but show determiners, conjunctions, and prepositions as separate leaves when the analysis requires them.

Grammar Categories Used

A noun phrase can be a single noun, a pronoun, or a determiner followed by a noun phrase. Examples in the ATIS sentences include “American airlines”, “a flight”, “five a.m.”, “six a.m.”, “Philadelphia”, “Atlanta”, and “the fare”. Proper names such as city names and airline names can be represented as proper-noun phrases.

A verb phrase contains the main verb and its complements. In “have a flight”, the verb have takes the noun phrase “a flight”. In “fly on American airlines”, the verb fly is followed by a prepositional phrase headed by on. In a question, an auxiliary such as does or is appears before the subject.

A prepositional phrase generally consists of a preposition followed by a noun phrase. Examples include “between five a.m. and six a.m.”, “on American airlines”, “from Atlanta”, “to Denver”, and “between Philadelphia and Atlanta”. Coordinated phrases may contain the conjunction and.

Tree / structural representation

> Phrase labels used below are standard syntactic categories. Lexical items such as airline names, city names, and clock times are grouped into suitable NP subtrees.

To verify a tree, read its leaf nodes from left to right. The resulting sequence must reproduce the original sentence exactly. Every internal node should dominate the words that form its constituent, and phrase boundaries should reflect the intended grammatical relationships.

When drawing the tree by hand, keep category labels aligned and avoid crossing branches. Group multiword names under a single NP where appropriate, but show determiners, conjunctions, and prepositions as separate leaves when the analysis requires them.

Sentence (a): Overall Structure

Sentence: “Does American airlines have a flight between five a.m. and six a.m.” This is a yes/no question. The auxiliary does is placed before the subject NP. The main predicate is the VP “have a flight between five a.m. and six a.m.”

The subject is “American airlines”, which functions as a noun phrase. The VP begins with the base-form verb have. Its direct object is “a flight”. The flight is further specified by a time-range phrase introduced by between.

The time range contains two coordinated time expressions: “five a.m.” and “six a.m.”. The conjunction and joins the two endpoints. In a detailed tree, the between phrase can be represented as a PP containing a coordinated NP or a specialized time-range constituent.

Tree / structural representation

> Constituent outline: [S [Aux Does] [NP American airlines] [VP have [NP a flight] [PP between [TimeRange five a.m. and six a.m.]]]].

The temporal modifier narrows the requested flight set by specifying a departure-time interval. The word between introduces two endpoints, and and coordinates those endpoints. An ATIS application can normalize five a.m. and six a.m. into standard time values before querying a flight database.

The auxiliary does carries the tense and forms the yes/no question. The main verb remains in its base form have. This separation is important in English question formation: the auxiliary appears before the subject, while the lexical verb follows the subject.

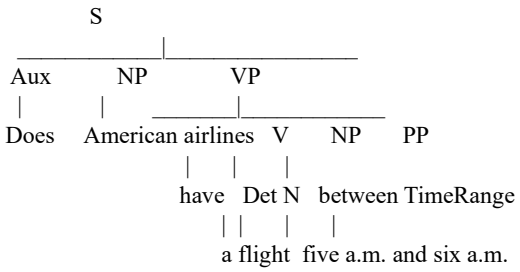
Sentence (a): Parse Tree

The tree begins with S. The auxiliary does is a child of the question structure, followed by the subject NP and the predicate VP. The VP contains the verb have, the object NP, and the PP that restricts the flight by departure-time interval.

The phrase American airlines is treated as a proper-name NP. The noun flight is the head of the object NP, and a is its determiner. The phrase between five a.m. and six a.m. is attached to the VP because it specifies the time of the flight.

The structure makes the grammatical roles clear: American airlines is the subject, have is the predicate head, a flight is the object, and the between phrase is a temporal modifier.

Tree / structural representation



The temporal modifier narrows the requested flight set by specifying a departure-time interval. The word between introduces two endpoints, and and coordinates those endpoints. An ATIS application can normalize five a.m. and six a.m. into standard time values before querying a flight database.

The auxiliary does carries the tense and forms the yes/no question. The main verb remains in its base form have. This separation is important in English question formation: the auxiliary appears before the subject, while the lexical verb follows the subject.

Sentence (b): Overall Structure

Sentence: “I would like to fly on American airlines.” This is a declarative sentence expressing a preference or request. The subject is the pronoun I. The predicate begins with the modal would, followed by like.

The verb like takes an infinitival VP complement: “to fly on American airlines”. The infinitive marker to introduces the verb fly. The PP “on American airlines” modifies fly and identifies the airline on which the speaker wants to travel.

A layered representation is useful: the outer VP is “would like [VP to fly [PP on American airlines]]”. The complement relationship between like and the infinitival phrase is important; the PP is attached to fly rather than to the subject.

Tree / structural representation

Bracketed structure: [S [NP I] [VP would [VP like [VP to [VP fly [PP on [NP American airlines]]]]]]].

The expression would like is a polite way of stating a preference. The verb like selects an infinitival complement because the desired action is expressed by fly. A dialogue system can interpret this as a request to search for air travel rather than as a statement about an existing flight.

The airline name is a multiword named entity. Although it consists of two words, it functions as one carrier name. A practical system can mark the complete span as an airline entity while retaining the NP label in the syntactic tree.

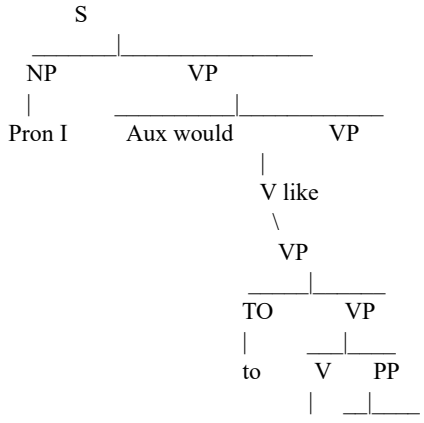
Sentence (b): Parse Tree

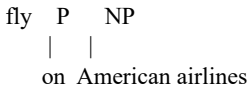
The root S branches into the subject NP and the VP. The subject NP consists of the pronoun I. The VP contains the modal auxiliary would and the lexical verb like.

The verb like selects a complement expressing the desired action. This complement is an infinitival VP beginning with to and containing fly. The PP on American airlines is a modifier of the flying event.

The tree therefore captures both the speaker’s intention and the airline information. It also shows why the phrase “on American airlines” belongs with fly: it describes the mode or carrier of the requested flight.

Tree / structural representation





The expression would like is a polite way of stating a preference. The verb like selects an infinitival complement because the desired action is expressed by fly. A dialogue system can interpret this as a request to search for air travel rather than as a statement about an existing flight.

The airline name is a multiword named entity. Although it consists of two words, it functions as one carrier name. A practical system can mark the complete span as an airline entity while retaining the NP label in the syntactic tree.

Sentence (c): Overall Structure

Sentence: “Please repeat that.” This is an imperative sentence. Imperatives commonly omit their subject; the understood subject is “you”. The sentence begins with please, a politeness marker, followed by the verb phrase “repeat that”.

The verb repeat is transitive in this sentence because it takes a direct object. The pronoun that is the object NP. The word please can be represented as an adverbial or politeness marker attached to the clause or VP.

A simplified parse tree may include an implicit subject NP (you), even though it is not pronounced. The surface tree can also omit the silent subject and begin directly with the politeness marker and VP. Both conventions are defensible if stated.

Tree / structural representation

Bracketed structure: [S [PoliteMarker Please] [VP [V repeat] [NP [Pron that]]]]. Understood subject: (you).

Imperatives are common in dialogue systems because users issue commands such as repeat that, list the flights, or show the fares. The omitted subject is normally understood as you, referring to the listener or system. The dialogue manager can map the command to an action and use the object to identify the information to be repeated.

Please is a politeness marker and does not change the argument structure of repeat. A grammar may attach it to the clause, attach it to the VP, or represent it with a separate pragmatic node. The important syntactic relation is that repeat takes that as its direct object.

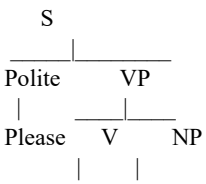
Sentence (c): Parse Tree

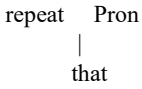
The root is an imperative S. Please is a discourse or politeness marker. The VP consists of the verb repeat and its object NP that. Since the object is a pronoun, it can be represented as a one-word NP.

The verb repeat requires something to be repeated. The word that supplies this object. The phrase has no overt determiner because that itself functions as a demonstrative pronoun in this usage.

The tree is simple, but it illustrates that not every sentence has an overt subject. Imperative constructions are a common case in which the subject is understood from context.

Tree / structural representation





(Implicit subject: you)

Imperatives are common in dialogue systems because users issue commands such as repeat that, list the flights, or show the fares. The omitted subject is normally understood as you, referring to the listener or system. The dialogue manager can map the command to an action and use the object to identify the information to be repeated.

Please is a politeness marker and does not change the argument structure of repeat. A grammar may attach it to the clause, attach it to the VP, or represent it with a separate pragmatic node. The important syntactic relation is that repeat takes that as its direct object.

Sentence (d): Overall Structure

Sentence: “Does American 487 have a first class section?” This is a yes/no question with does as the auxiliary. The subject NP is “American 487”, which refers to a particular airline flight.

The main VP is “have a first class section”. The verb have takes the object NP “a first class section”. Within that NP, a is the determiner, first class is a compound modifier, and section is the head noun.

The question structure places does before the subject. In a tree that separates syntax from surface word order, the auxiliary may be represented under a question or tense phrase; for this assignment, the simpler S–Aux–NP–VP representation is sufficient.

Tree / structural representation

Bracketed structure: [S [Aux Does] [NP [Name American] [Number 487]] [VP [V have] [NP [Det a] [Adj/Compound first class] [N section]]]].

The number 487 is not an ordinary quantity in this context. Together with American it identifies a particular flight. A domain-specific recognizer can classify the span as a flight identifier and preserve it as one semantic slot.

First class functions as a compound modifier. The words jointly identify the cabin category, while section is the head noun. The determiner a marks the noun phrase as indefinite. This structure allows the system to interpret the question as asking whether the flight offers a first-class section.

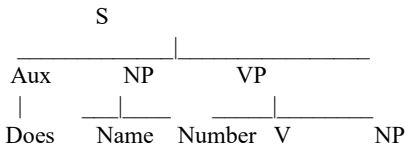
Sentence (d): Parse Tree

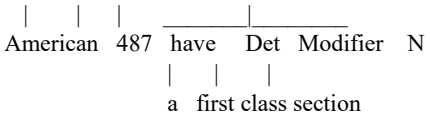
The subject is a noun phrase containing the airline identifier American 487. American functions as a proper name and 487 as a flight number. They are grouped together because they jointly identify the flight.

The object noun phrase is headed by section. The phrase first class modifies section and expresses the cabin category. The determiner a marks the noun phrase as indefinite.

The tree shows the auxiliary, subject, predicate, and object distinctly. It is useful for ATIS because the parser can extract the flight identifier and the requested attribute (whether a first-class section is available).

Tree / structural representation





The number 487 is not an ordinary quantity in this context. Together with American it identifies a particular flight. A domain-specific recognizer can classify the span as a flight identifier and preserve it as one semantic slot.

First class functions as a compound modifier. The words jointly identify the cabin category, while section is the head noun. The determiner a marks the noun phrase as indefinite. This structure allows the system to interpret the question as asking whether the flight offers a first-class section.

Sentence (e): Overall Structure

Sentence: “I need to fly between Philadelphia and Atlanta.” This is a declarative request. The subject is I, and the VP begins with need. The verb need takes an infinitival VP complement “to fly between Philadelphia and Atlanta”.

The infinitive marker to is understood in the surface sentence only if the grammar permits bare infinitival complements after need; in this sentence, the actual words are “need to fly”. Therefore to is an overt infinitive marker between need and fly.

The PP between Philadelphia and Atlanta modifies fly and specifies the route endpoints. The two city names are coordinated by and. The route phrase is important for extracting origin/destination information from the request.

Tree / structural representation

Bracketed structure: [S [NP I] [VP need [VP to [VP fly [PP between [CoordNP Philadelphia and Atlanta]]]]]].

The verb need expresses a requirement and takes an infinitival complement. The phrase to fly names the action the speaker needs to perform. The route phrase gives the two locations associated with that action.

The construction between Philadelphia and Atlanta names a pair of route endpoints without explicitly assigning an origin and destination direction. A system may preserve both locations as a route pair or use dialogue context to determine direction.

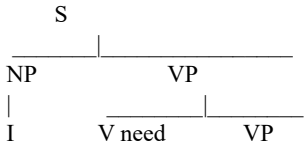
Sentence (e): Parse Tree

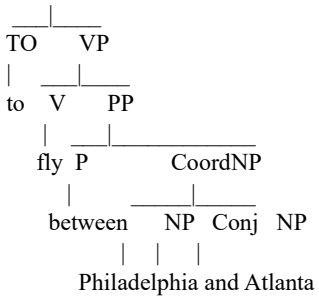
The outer sentence contains the NP I and a VP headed by need. The complement of need is an infinitival VP. The word to marks the infinitive and fly is the main verb.

The verb fly is modified by the PP between Philadelphia and Atlanta. The PP contains a coordinated pair of city-name NPs. The conjunction and joins the two endpoints.

For semantic interpretation, the parser may map the first city to the starting point and the second city to the destination, because the construction “between X and Y” gives a route pair rather than a directional from-to relation.

Tree / structural representation





The verb need expresses a requirement and takes an infinitival complement. The phrase to fly names the action the speaker needs to perform. The route phrase gives the two locations associated with that action.

The construction between Philadelphia and Atlanta names a pair of route endpoints without explicitly assigning an origin and destination direction. A system may preserve both locations as a route pair or use dialogue context to determine direction.

Sentence (f): Overall Structure

Sentence: “What is the fare from Atlanta to Denver?” This is a wh-question. The wh-word what appears at the beginning and asks for information about the fare. The copular verb is links the subject or predicate nominal fare with the requested information.

The noun phrase the fare is headed by fare and introduced by the determiner the. The PP from Atlanta to Denver specifies the route for which the fare is requested. It contains two prepositions and two city-name NPs.

A simplified tree places the wh-phrase What at the beginning, followed by the copular clause. Some formal grammars analyze what as the fronted complement of is, while the fare phrase is the subject. The tree below uses a clear surface-oriented representation.

Tree / structural representation

Bracketed structure: [S [WHNP What] [VP is [NP the fare [PP from Atlanta [PP to Denver]]]]].

A wh-question requests a value rather than a yes/no confirmation. Here, what asks for the fare amount, while from Atlanta to Denver supplies the route context needed to retrieve that amount.

In a semantic frame, the sentence can be represented as intent=FIND_FARE, origin=Atlanta, and destination=Denver. The parse tree supports this representation by grouping the fare noun and the route modifiers into identifiable constituents.

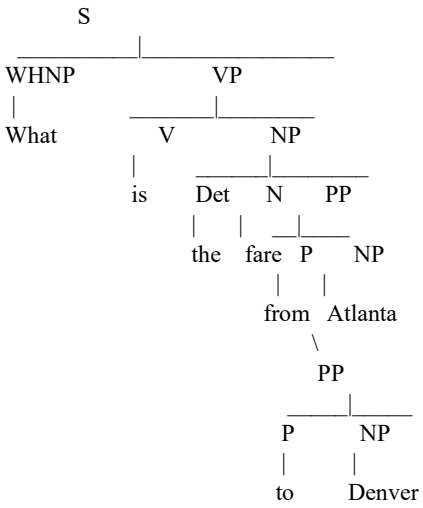
Sentence (f): Parse Tree

The root S has the fronted wh-phrase What and the remaining clause. The word is is the copular verb. The NP the fare is the nominal expression whose value is being asked about.

The route PP begins with from and contains the city Atlanta. A second PP beginning with to contains Denver. This nested representation expresses the direction from Atlanta to Denver.

The tree is useful for an ATIS application because it separates the query type (fare) from the route parameters (origin Atlanta and destination Denver).

Tree / structural representation



A wh-question requests a value rather than a yes/no confirmation. Here, what asks for the fare amount, while from Atlanta to Denver supplies the route context needed to retrieve that amount.

In a semantic frame, the sentence can be represented as intent=FIND_FARE, origin=Atlanta, and destination=Denver. The parse tree supports this representation by grouping the fare noun and the route modifiers into identifiable constituents.

Sentence (g): Overall Structure

Sentence: “Is there an American airlines flight from Philadelphia to Dallas?” This is a yes/no existential question. The auxiliary or copular verb is appears first, followed by the existential expression there and the noun phrase “an American airlines flight”.

The noun phrase has an as determiner, American airlines as a compound proper-name modifier, and flight as the head noun. The PP from Philadelphia to Dallas modifies flight and specifies the route.

The sentence asks whether at least one flight exists for the specified airline and route. The tree should therefore preserve the existential there construction and attach the route phrase to the flight NP.

Tree / structural representation

Bracketed structure: [S [Aux Is] [Expl there] [NP an [NP American airlines] flight [PP from Philadelphia [PP to Dallas]]]].

Existential there introduces an assertion or question about the existence of an entity. It is not a locative adverb in this sentence. The question asks whether a flight exists, not whether an object is physically located at a place.

The noun phrase contains the airline name and the head noun flight. The route phrase specifies the flight's route. A semantic system can therefore form a query for an American Airlines flight from Philadelphia to Dallas.

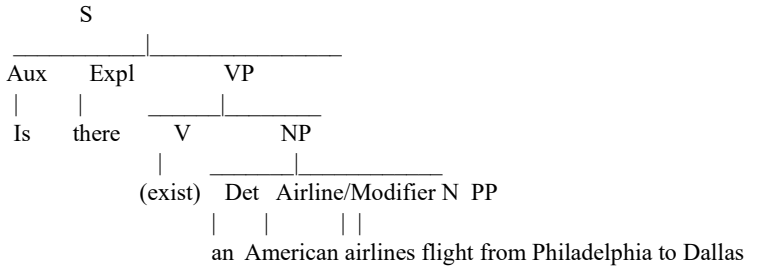
Sentence (g): Parse Tree

The surface order begins with Is, which signals a yes/no question. There is an expletive or existential subject-like element; it does not refer to a physical location in this sentence.

The post-verbal NP is an American airlines flight. Its head is flight. The airline phrase identifies the carrier, while the route PP identifies the endpoints Philadelphia and Dallas.

The tree makes the main information slots visible: existence question, airline, flight entity, origin, and destination. This is exactly the type of structure that a spoken air-travel information system needs to interpret.

Tree / structural representation



Existential there introduces an assertion or question about the existence of an entity. It is not a locative adverb in this sentence. The question asks whether a flight exists, not whether an object is physically located at a place.

The noun phrase contains the airline name and the head noun flight. The route phrase specifies the flight's route. A semantic system can therefore form a query for an American Airlines flight from Philadelphia to Dallas.

Comparison of the Seven Trees

The seven sentences illustrate several common ATIS syntactic patterns. Sentences (a), (d), and (g) are yes/no questions. Sentence (f) is a wh-question. Sentences (b) and (e) are declarative requests or statements. Sentence (c) is an imperative.

The most common phrase types are NP, VP, and PP. NPs identify airlines, flights, cities, times, and objects. VPs express actions or relations such as have, fly, need, like, and repeat. PPs express route, carrier, and temporal information.

The parse trees also show that surface word order differs across question types. Yes/no questions place an auxiliary before the subject, while wh-questions front the information-seeking expression. Imperatives may omit an overt subject.

Tree / structural representation

> Key patterns: yes/no = Aux + subject + predicate; wh-question = WH phrase + clause; declarative = subject + predicate; imperative = optional politeness marker + VP.

To verify a tree, read its leaf nodes from left to right. The resulting sequence must reproduce the original sentence exactly. Every internal node should dominate the words that form its constituent, and phrase boundaries should reflect the intended grammatical relationships.

When drawing the tree by hand, keep category labels aligned and avoid crossing branches. Group multiword names under a single NP where appropriate, but show determiners, conjunctions, and prepositions as separate leaves when the analysis requires them.

Role of Constituency in ATIS

Constituency groups words that function together as a unit. In “American airlines”, the two words form an NP referring to a carrier. In “a flight”, the determiner and noun form an NP. In “from Atlanta to Denver”, the prepositions introduce route information.

A parser can use constituent boundaries to identify slots. It can detect the airline, city names, departure times, cabin class, and requested fare. For example, the PP from Atlanta to Denver can be interpreted as an origin-destination pair.

The parse tree is not merely a diagram. It provides a structural representation that downstream semantic interpretation can use. The same words can have different meanings depending on their syntactic attachment, so correct phrase grouping matters.

Tree / structural representation

Constituency supports semantic slot filling: airline, flight number, route, time, cabin class, fare, and user action.

To verify a tree, read its leaf nodes from left to right. The resulting sequence must reproduce the original sentence exactly. Every internal node should dominate the words that form its constituent, and phrase boundaries should reflect the intended grammatical relationships.

When drawing the tree by hand, keep category labels aligned and avoid crossing branches. Group multiword names under a single NP where appropriate, but show determiners, conjunctions, and prepositions as separate leaves when the analysis requires them.

Attachment and Coordination

Prepositional phrase attachment is important in ATIS. A PP can modify a verb, a noun, or an entire clause. In “fly on American airlines”, the PP modifies the flying event. In “a flight from Philadelphia to Dallas”, the PP modifies the noun flight.

Coordination occurs in “five a.m. and six a.m.” and “Philadelphia and Atlanta”. The conjunction and joins two parallel constituents. A parse tree should place the coordinated expressions under a common coordination node.

A simple grammar may not distinguish every possible attachment. More detailed grammars use semantic constraints and lexical information. For this assignment, the attachment choices shown are the most natural interpretations of the given sentences.

Tree / structural representation

Coordination: [CoordP [NP Philadelphia] [Conj and] [NP Atlanta]]. Route PP: [PP between [CoordP ...]].

To verify a tree, read its leaf nodes from left to right. The resulting sequence must reproduce the original sentence exactly. Every internal node should dominate the words that form its constituent, and phrase boundaries should reflect the intended grammatical relationships.

When drawing the tree by hand, keep category labels aligned and avoid crossing branches. Group multiword names under a single NP where appropriate, but show determiners, conjunctions, and prepositions as separate leaves when the analysis requires them.

Limitations and Parsing Notes

The trees are simplified phrase-structure analyses. A full linguistic analysis may include tense phrases, complementizer phrases, wh-movement, silent subjects, and traces. For an introductory NLP assignment, S, NP, VP, PP, and lexical categories provide a readable representation.

Some ATIS expressions are multiword named entities. “American airlines” should be recognized as one carrier name, and “five a.m.” as one time expression. A practical system often combines syntactic parsing with named-entity recognition to avoid splitting these expressions incorrectly.

The tree for “Is there…” uses an existential construction. The word there is not a locative adverb in this context. Similarly, “Please repeat that” is imperative, with an understood subject you. These distinctions improve semantic interpretation.

Tree / structural representation

Tree drawing tip: place S at the top, phrase categories below it, and actual sentence words on the bottom row. Draw one branch per immediate constituent.

Conclusion for Question 1

The seven ATIS sentences have been analyzed using phrase-structure trees. The trees identify sentence type, subject, predicate, noun phrases, verb phrases, prepositional phrases, coordination, and important lexical heads.

The analyses demonstrate how a parser can represent questions about airline identity, flight number, time, route, fare, and cabin class. The resulting trees support later semantic interpretation and extraction of structured travel information.

In conclusion, syntactic tree structures are a useful foundation for ATIS language understanding. They convert a linear sentence into a hierarchy of grammatical units and make the relationships among words explicit.

Tree / structural representation

All seven sentence trees are provided in bracketed and/or ASCII form above. They can be redrawn as conventional node-and-branch diagrams for submission.

Introduction to the Earley Parser

The Earley parser is a chart-parsing algorithm used to determine whether a string can be generated by a context-free grammar. It is especially useful when a grammar is ambiguous or contains left recursion. The algorithm processes the input from left to right and stores partial parsing information in chart sets.

Each chart state records a grammar production, a dot showing how much of the production has been recognized, and the input positions at which the constituent begins and ends. The three central operations are predictor, scanner, and completer.

The input for this problem is “Christ Opened the eyes”. The grammar contains the lexical rule V → opened, so the capitalized surface token Opened is normalized to lowercase opened before parsing. This is a standard case-normalization step.

Derivation / chart / tree

State notation: [A → α • β, i] means that production A → αβ has recognized α, expects β, began at input position i, and is currently in chart position j.

The chart is the central data structure of the Earley algorithm. Each chart column corresponds to a boundary between input tokens. A state records a partial production, the position of its dot, and the input position at which the constituent began. This origin value allows completed phrases to connect with earlier expectations.

Unlike a parser that commits immediately to one interpretation, Earley parsing preserves alternative hypotheses. The algorithm stores each distinct state once, so repeated prediction does not cause endless duplication. This makes it suitable for grammars containing recursion and ambiguity.

Given Grammar

The grammar contains sentence, noun phrase, verb phrase, lexical, determiner, and prepositional phrase rules. The relevant rules are S → NP VP; VP → V NP; NP → N; NP → Det NP; V → opened; N → Christ; Det → the; and N → eyes.

Other productions in the supplied grammar include VP → V, VP → V NP PP, V → ate, NP → NP PP, NP → NP NP, PP → Prep NP, N → blind, N → bread, Prep → of, and Prep → in. These rules are retained even though not all are needed for the input.

For clarity, the lexical rule N → Christ treats Christ as a proper noun. The input word Opened is matched to V → opened after case normalization. The phrase "the eyes" is built using NP → Det NP and NP → N.

Derivation / chart / tree

Relevant productions:
S → NP VP
NP → N | Det NP | NP PP | NP NP
VP → V | V NP | V NP PP
V → opened | ate
N → Christ | blind | eyes | bread
Det → the | a
PP → Prep NP
Prep → of | in

Production group	Rules
Sentence	S → NP VP
Noun phrase	NP → N | Det NP | NP PP | NP NP
Verb phrase	VP → V | V NP | V NP PP
Lexical	V → opened | ate; N → Christ | blind | eyes | bread
Other	Det → the | a; PP → Prep NP; Prep → of | in

The chart is the central data structure of the Earley algorithm. Each chart column corresponds to a boundary between input tokens. A state records a partial production, the position of its dot, and the input position at which the constituent began. This origin value allows completed phrases to connect with earlier expectations.

Unlike a parser that commits immediately to one interpretation, Earley parsing preserves alternative hypotheses. The algorithm stores each distinct state once, so repeated prediction does not cause endless duplication. This makes it suitable for grammars containing recursion and ambiguity.

Input Tokenization and Positions

The input sentence is tokenized into four words: Christ, Opened, the, eyes. After normalization, the token sequence is Christ, opened, the, eyes. The chart has five positions: 0 before the first token, 1 after Christ, 2 after opened, 3 after the, and 4 after eyes.

Chart set C0 contains states that begin at position 0. C1 contains states after scanning the first token. C2 is after the second token, C3 after the third token, and C4 after the fourth token.

The goal is to derive S from the complete input span [0,4]. A successful parse must contain a completed start-symbol state in C4, with the dot at the end of S → NP VP and origin 0.

Derivation / chart / tree

Position: 0 1 2 3 4
Input: Christ opened the eyes
Chart: C0 C1 C2 C3 C4

A practical implementation can store each state as a tuple containing the production, dot position, and origin index. The chart is a list of sets, one for each input boundary. Prediction, scanning, and completion are applied until no new states can be added.

The final acceptance test checks whether the last chart column contains a completed start-symbol item whose origin is zero. This proves that the grammar has recognized the complete input rather than only a prefix.

Earley State Format

An Earley item is written as [A → α • β, i]. A is the left-hand nonterminal. α is the portion already recognized, and β is the remaining portion to be recognized. The dot moves from left to right as the parser consumes or predicts constituents.

The origin i records where the constituent began. The current chart column gives the end position. For example, [S → • NP VP, 0] means that the parser expects an NP followed by a VP, beginning at position 0.

A completed item has the dot at the far right. For example, [NP → N •, 0] means that an NP beginning at 0 has been recognized. The completer uses such completed items to advance states that were waiting for that NP.

Derivation / chart / tree

Examples:
[S → • NP VP, 0] incomplete; expects NP then VP
[NP → N •, 0] completed NP
[VP → V • NP, 1] VP has recognized V and expects NP

The chart is the central data structure of the Earley algorithm. Each chart column corresponds to a boundary between input tokens. A state records a partial production, the position of its dot, and the input position at which the constituent began. This origin value allows completed phrases to connect with earlier expectations.

Unlike a parser that commits immediately to one interpretation, Earley parsing preserves alternative hypotheses. The algorithm stores each distinct state once, so repeated prediction does not cause endless duplication. This makes it suitable for grammars containing recursion and ambiguity.

Earley Operation 1: Predictor

The predictor is applied when the symbol immediately after the dot is a nonterminal. It adds chart items for every production whose left-hand side is that nonterminal. For example, from [S → • NP VP, 0], the parser predicts all NP productions.

The grammar has several NP rules: NP → N, NP → Det NP, NP → NP PP, and NP → NP NP. Therefore, the predictor adds these alternatives to the current chart set. The dot begins at the right-hand side's first position.

Prediction does not consume an input word. It simply records possible grammatical structures that may satisfy the expected nonterminal. This is why predicted items remain in the same chart column.

Derivation / chart / tree

Predictor example:
[S → • NP VP, 0]
produces [NP → • N, 0]
    [NP → • Det NP, 0]
    [NP → • NP PP, 0]
    [NP → • NP NP, 0]

Prediction is a top-down operation because it uses the grammar to anticipate which category may occur next. It does not consume an input token. If several productions expand the expected nonterminal, each production is added as a separate state in the current chart column.

The chart can therefore contain several competing possibilities before the next word is scanned. Later scanning and completion steps determine which possibilities are compatible with the actual input. This is one reason the Earley algorithm can handle ambiguous context-free grammars.

Earley Operation 2: Scanner

The scanner is used when the symbol after the dot is a terminal. It compares that terminal with the next input token. If they match, it advances the dot and moves the resulting item to the next chart column.

For example, [N → • Christ, 0] can scan the first input token Christ and produce [N → Christ •, 0] in C1. A scanner step consumes exactly one input token.

The scanner does not apply to nonterminals. If the symbol after the dot is NP or VP, the predictor or completer is used instead. This separation makes the algorithm systematic.

Derivation / chart / tree

Scanner example:
C0: [N → • Christ, 0]
Input[0] = Christ
C1: [N → Christ •, 0]

Scanning is the operation that advances the input position. A terminal symbol immediately after the dot is compared with the next input token. If they match, the dot moves one place to the right and the advanced state is placed in the next chart column.

The lexical rules are case-sensitive as written, so the input token Opened must be normalized to opened or matched using a case-insensitive lexical lookup. Without that normalization, the rule V → opened would not match the capitalized token.

Earley Operation 3: Completer

The completer is applied when an item becomes complete. It looks back to the chart column where the completed constituent began and finds items that were waiting for that constituent. Those waiting items are advanced over the completed nonterminal.

For example, when [NP → N •, 0] is completed in C1, the parser checks C0 for items expecting NP. The initial item [S → • NP VP, 0] is advanced to [S → NP • VP, 0] in C1.

Completion therefore combines recognized constituents into larger constituents. It does not consume a word; it advances a dot in an item that was waiting for the completed phrase.

Derivation / chart / tree

Completer example:
C0: [S → • NP VP, 0]
C1: [NP → N •, 0]
C1 receives: [S → NP • VP, 0]

Completion is a bottom-up operation because it uses a constituent that has already been recognized. The parser looks back to the chart column where the constituent began and advances every state that was waiting for that category.

The origin index is essential. A completed NP beginning at position 2 should advance a VP item that expects an NP beginning at position 2. It must not advance an unrelated state that expects an NP at another input position.

Initialization at Chart Set C0

The parser begins by adding the augmented start production S' → S. The initial item is [S' → • S, 0]. Because the dot is before S, the predictor adds the grammar production S → NP VP.

The predicted sentence item is [S → • NP VP, 0]. Since NP is next, the predictor adds all NP productions. The parser also predicts lexical N and determiner rules when those nonterminals become expected.

At this stage, no input has been consumed. All states in C0 have origin 0 and current position 0. The chart represents the set of grammatical possibilities that can begin a sentence.

Derivation / chart / tree

Initial C0 core:
[S' → • S, 0]
[S → • NP VP, 0]
[NP → • N, 0]
[NP → • Det NP, 0]
[NP → • NP PP, 0]
[NP → • NP NP, 0]

A practical implementation can store each state as a tuple containing the production, dot position, and origin index. The chart is a list of sets, one for each input boundary. Prediction, scanning, and completion are applied until no new states can be added.

The final acceptance test checks whether the last chart column contains a completed start-symbol item whose origin is zero. This proves that the grammar has recognized the complete input rather than only a prefix.

Processing the First Word: Christ

The parser expects an NP at the beginning of the sentence. One predicted path is NP → N, followed by N → Christ. The lexical state [N → • Christ, 0] matches the first token Christ.

The scanner advances the lexical item into C1 as [N → Christ •, 0]. The completer then uses NP → N to create [NP → N •, 0] in C1. This is a completed noun phrase spanning the first word.

The completed NP advances the sentence item that was waiting for an NP. Thus C1 includes [S → NP • VP, 0]. The parser now expects a VP beginning at position 1.

Derivation / chart / tree

Key C1 items:
[N → Christ •, 0]
[NP → N •, 0]
[S → NP • VP, 0]

Scanning is the operation that advances the input position. A terminal symbol immediately after the dot is compared with the next input token. If they match, the dot moves one place to the right and the advanced state is placed in the next chart column.

The lexical rules are case-sensitive as written, so the input token Opened must be normalized to opened or matched using a case-insensitive lexical lookup. Without that normalization, the rule V → opened would not match the capitalized token.

Processing the Second Word: opened

At C1, the sentence item [S → NP • VP, 0] expects VP. The predictor adds VP productions, including VP → V, VP → V NP, and VP → V NP PP. It then predicts the lexical possibilities for V.

The grammar contains V → opened and V → ate. The input token at position 1 is opened, so the scanner advances [V → • opened, 1] into C2 as [V → opened •, 1].

The completed V can advance VP items that were waiting for V. In particular, VP → V becomes complete, while VP → V NP advances to [VP → V • NP, 1]. The latter path is needed because the remaining input contains “the eyes”, which can form an NP.

Derivation / chart / tree

Important C2 items:
[V → opened •, 1]
[VP → V •, 1]
[VP → V • NP, 1]
[VP → V • NP PP, 1]

Scanning is the operation that advances the input position. A terminal symbol immediately after the dot is compared with the next input token. If they match, the dot moves one place to the right and the advanced state is placed in the next chart column.

The lexical rules are case-sensitive as written, so the input token Opened must be normalized to opened or matched using a case-insensitive lexical lookup. Without that normalization, the rule V → opened would not match the capitalized token.

Predicting the Object NP

The item [VP → V • NP, 1] expects an NP beginning at position 2. The predictor therefore adds the NP productions to C2 with origin 2. These include NP → N, NP → Det NP, NP → NP PP, and NP → NP NP.

The next input token is the, which matches the determiner production Det → the. The parser predicts [Det → • the, 2] and scans it into C3 as [Det → the •, 2].

When the determiner is complete, it advances [NP → • Det NP, 2] to [NP → Det • NP, 2]. This item now expects another NP, which can be supplied by the noun eyes through NP → N.

Derivation / chart / tree

After scanning “the” into C3:
[Det → the •, 2]
[NP → Det • NP, 2]
[NP → • N, 3] (predicted for the nested NP)

A practical implementation can store each state as a tuple containing the production, dot position, and origin index. The chart is a list of sets, one for each input boundary. Prediction, scanning, and completion are applied until no new states can be added.

The final acceptance test checks whether the last chart column contains a completed start-symbol item whose origin is zero. This proves that the grammar has recognized the complete input rather than only a prefix.

Processing the Final Word: eyes

The parser is now expecting an NP after the determiner. One predicted route is NP → N, followed by N → eyes. The lexical item [N → • eyes, 3] matches the fourth input token eyes.

The scanner advances the lexical item into C4 as [N → eyes •, 3]. The completer uses NP → N to produce [NP → N •, 3]. This completes the embedded NP that follows the determiner.

The completed embedded NP advances the outer determiner phrase [NP → Det • NP, 2] to [NP → Det NP •, 2]. The result is the completed object NP “the eyes”, spanning positions 2 through 4.

Derivation / chart / tree

Key C4 / completion items:
[N → eyes •, 3]
[NP → N •, 3]
[NP → Det NP •, 2]

Scanning is the operation that advances the input position. A terminal symbol immediately after the dot is compared with the next input token. If they match, the dot moves one place to the right and the advanced state is placed in the next chart column.

The lexical rules are case-sensitive as written, so the input token Opened must be normalized to opened or matched using a case-insensitive lexical lookup. Without that normalization, the rule V → opened would not match the capitalized token.

Completing the Verb Phrase

The completed object NP “the eyes” began at position 2 and ends at position 4. In C2, the item [VP → V • NP, 1] was waiting for an NP that began at position 2. The completer advances it to [VP → V NP •, 1] in C4.

This completed VP spans “opened the eyes”, from position 1 to position 4. It is the predicate required by the sentence production S → NP VP.

The completed VP is then used to advance the sentence item [S → NP • VP, 0] from C1. Since that item began at 0 and expects a VP beginning at 1, the completed VP satisfies the expectation.

Derivation / chart / tree

Completed VP:
[VP → V NP •, 1]
Span: tokens 1–4 = “opened the eyes”

Completion is a bottom-up operation because it uses a constituent that has already been recognized. The parser looks back to the chart column where the constituent began and advances every state that was waiting for that category.

The origin index is essential. A completed NP beginning at position 2 should advance a VP item that expects an NP beginning at position 2. It must not advance an unrelated state that expects an NP at another input position.

Completing the Sentence

The sentence item [S → NP • VP, 0] is waiting for a VP beginning at position 1. The completed VP [VP → V NP •, 1] satisfies this requirement. The parser therefore creates [S → NP VP •, 0] in C4.

The dot is at the end of the start production, and the item begins at position 0 and ends at position 4. Therefore the entire input string has been recognized as a sentence by the grammar.

The augmented start item [S' → S •, 0] can also be completed in C4. Its presence confirms successful parsing. The parser accepts the input.

Derivation / chart / tree

Acceptance item in C4:
[S → NP VP •, 0]
[S' → S •, 0]
Conclusion: ACCEPT

Completion is a bottom-up operation because it uses a constituent that has already been recognized. The parser looks back to the chart column where the constituent began and advances every state that was waiting for that category.

The origin index is essential. A completed NP beginning at position 2 should advance a VP item that expects an NP beginning at position 2. It must not advance an unrelated state that expects an NP at another input position.

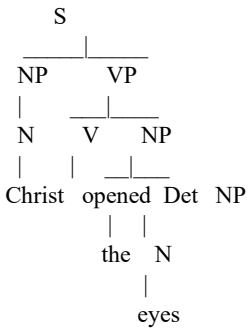
Constituency Tree for the Input

The successful derivation is S → NP VP. The subject NP is derived from NP → N and N → Christ. The VP is derived from VP → V NP and V → opened. The object NP is derived from NP → Det NP, with Det → the and the embedded NP → N → eyes.

The resulting tree contains one sentence node, two main constituents, and lexical leaves corresponding to the four input tokens. The tree demonstrates that “Christ” is the subject and “the eyes” is the object of “opened”.

The grammar therefore generates the complete sentence through a sequence of valid productions. The derivation also illustrates why the parser predicts several possible NP rules but only one successful path matches this particular input.

Derivation / chart / tree



The parse tree and the derivation provide complementary views of the same analysis. The tree displays hierarchy and parent-child relationships, while the derivation displays the order in which grammar productions are applied. Both must yield the same terminal sequence as the input.

The object phrase is built recursively: NP → Det NP, followed by NP → N. This structure groups the determiner the with the noun eyes. The recursive rule is a compact way to represent noun phrases containing determiners and nominal material.

Leftmost Derivation

A leftmost derivation expands the leftmost nonterminal at each step. Starting from S, apply S → NP VP. Expand the subject NP using NP → N, and then N → Christ.

Next expand VP using VP → V NP. Expand V using V → opened. Expand the object NP using NP → Det NP, then Det → the, NP → N, and N → eyes.

The final terminal sequence is Christ opened the eyes. This derivation provides a second way to verify the parse tree and the grammar's ability to generate the input.

Derivation / chart / tree

S
⇒ NP VP
⇒ N VP
⇒ Christ VP
⇒ Christ V NP
⇒ Christ opened NP
⇒ Christ opened Det NP
⇒ Christ the NP (with opened retained in sequence)
⇒ Christ opened the N
⇒ Christ opened the eyes

The parse tree and the derivation provide complementary views of the same analysis. The tree displays hierarchy and parent-child relationships, while the derivation displays the order in which grammar productions are applied. Both must yield the same terminal sequence as the input.

The object phrase is built recursively: NP → Det NP, followed by NP → N. This structure groups the determiner the with the noun eyes. The recursive rule is a compact way to represent noun phrases containing determiners and nominal material.

Chart Summary by Input Position

The chart summary records the most important completed constituents at each position. C0 contains the initial sentence expectation and predicted NP alternatives. C1 contains the completed subject NP and a sentence item waiting for VP.

C2 contains the completed verb opened and VP items that expect an object NP. C3 contains the scanned determiner the and an NP item waiting for its noun phrase. C4 contains the completed noun eyes, completed object NP, completed VP, and completed sentence.

The chart does not need to store every duplicate state more than once. A practical Earley implementation uses sets, so adding a state already present has no effect. This avoids infinite repetition when grammars contain left-recursive rules.

Derivation / chart / tree

Position | Main result
C0 | Initial S and predicted NP rules
C1 | Subject NP = Christ

C2 | Verb = opened; VP expects NP
C3 | Det = the; NP expects nested NP
C4 | Object NP = the eyes; VP and S complete

A practical implementation can store each state as a tuple containing the production, dot position, and origin index. The chart is a list of sets, one for each input boundary. Prediction, scanning, and completion are applied until no new states can be added.

The final acceptance test checks whether the last chart column contains a completed start-symbol item whose origin is zero. This proves that the grammar has recognized the complete input rather than only a prefix.

Why Other Grammar Rules Are Not Used

The grammar includes additional rules such as VP → V NP PP, VP → V, NP → NP PP, NP → NP NP, V → ate, N → blind, N → bread, Prep → of, and Prep → in. These rules create other possible sentence structures but are not needed to parse the current input.

The input contains no preposition, so PP → Prep NP and the preposition rules do not participate in the successful derivation. It contains neither ate nor bread nor blind, so those lexical alternatives do not match any input token.

A parser may predict alternatives before it knows which will succeed. The scanner eliminates alternatives that do not match the next word, and the completer advances only constituents that have been successfully recognized. This is how the chart algorithm handles a grammar with more rules than the input requires.

Derivation / chart / tree

Unused examples: PP → Prep NP; Prep → of/in; V → ate; N → blind/bread. They remain part of the grammar but are not on the successful parse path.

Chart set	Completed / important item
C0	[S' → • S, 0], [S → • NP VP, 0]
C1	[NP → N •, 0], [S → NP • VP, 0]
C2	[V → opened •, 1], [VP → V • NP, 1]
C3	[Det → the •, 2], [NP → Det • NP, 2]
C4	[NP → Det NP •, 2], [VP → V NP •, 1], [S → NP VP •, 0]

A practical implementation can store each state as a tuple containing the production, dot position, and origin index. The chart is a list of sets, one for each input boundary. Prediction, scanning, and completion are applied until no new states can be added.

The final acceptance test checks whether the last chart column contains a completed start-symbol item whose origin is zero. This proves that the grammar has recognized the complete input rather than only a prefix.

Ambiguity and Grammar Behavior

The supplied grammar contains left-recursive rules such as NP → NP PP and NP → NP NP. Earley parsing can handle left recursion because it stores chart states and avoids repeatedly adding the same item. This is one advantage over naive top-down recursive-descent parsing.

The grammar may generate multiple parse trees for longer inputs. For this particular four-word input, the successful structure is straightforward: Christ is an NP, opened is a verb, and the eyes is the object NP. The extra grammar rules do not create a competing complete parse for these tokens.

If the grammar were expanded with additional lexical or structural productions, ambiguity could arise. The chart would retain multiple states and potentially multiple derivations. A parser can then return one parse, all parses, or a packed parse forest.

Derivation / chart / tree

> Important property: Earley parsing supports left-recursive and ambiguous context-free grammars while keeping partial results in chart sets.

A practical implementation can store each state as a tuple containing the production, dot position, and origin index. The chart is a list of sets, one for each input boundary. Prediction, scanning, and completion are applied until no new states can be added.

The final acceptance test checks whether the last chart column contains a completed start-symbol item whose origin is zero. This proves that the grammar has recognized the complete input rather than only a prefix.

Complexity and Practical Uses

The Earley algorithm has a general worst-case time complexity of $O(n^3)$ for an input of length n. For unambiguous grammars, it can run in $O(n^2)$, and for many practical deterministic grammars it can run in linear time. Its space use is generally polynomial because chart items are stored across input positions.

Earley parsing is useful in NLP because natural-language grammars often contain optional constituents, recursion, and ambiguity. It can be used for syntactic analysis, grammar checking, and language understanding systems.

The algorithm is particularly suitable for this exercise because it makes the parsing process explicit. Predictor, scanner, and completer operations can be traced in a table, showing exactly how each word contributes to a larger constituent.

Derivation / chart / tree

> Operations recap: Predictor adds rules for expected nonterminals; Scanner consumes matching terminals; Completer advances items waiting for completed constituents.

Conclusion for Question 2

The Earley parser successfully recognizes the input “Christ Opened the eyes” after normalizing Opened to opened. The subject NP is “Christ”, the verb is “opened”, and the object NP is “the eyes”.

The successful chart contains the completed start item [S → NP VP •, 0] in C4. The corresponding parse tree is S(NP(N(Christ)), VP(V(opened), NP(Det(the), NP(N(eyes))))).

In conclusion, the Earley algorithm provides a systematic method for parsing context-free grammars. Its predictor, scanner, and completer operations build constituents incrementally and confirm that the input can be generated by the given grammar.

Derivation / chart / tree

> Final result: ACCEPT. Parse: [S [NP [N Christ]] [VP [V opened] [NP [Det the] [NP [N eyes]]]]].